



COURSE OUTLINE: PRELIMS

1. Programming Language
2. Intro to C++
3. C++ Sample Code
4. C++ Code Part
5. C++ Basic Concept
6. Getting the Tools
7. Printing a Text
8. Comments
9. Variables and Value
10. Working with Variables

Introduction to C++

PROGRAMMING LANGUAGE

What is Programming Language and Example of it.

- Programming Language - Is a standardized communication technique for expressing Instructions to a computer.
 - Examples: CH, C, CB, Python, Java
- Categories of Programming Language:
 - High-Level Programming Language uses a more natural and human-readable syntax which makes it easier for humans to understand and write
 - Low-level Assembly Language users write code which is almost identical to the one computer understands.

C++

What is C++?

- C++ is a general-purpose programming language Invented in the early 1980s by Bjarne Stroustrup at Bell Labs.
- The purpose of C++ is to precisely define a series of operations that a computer can perform to accomplish a task. Most of these operations involve manipulating numbers and text, but anything that the computer can physically do can be programmed in C++.

Why Use C++?

- C++ is one of the world's most popular programming languages.
- C++ can be found in today's operating systems, Graphical User Interfaces, and embedded systems.
- C++ is an object-oriented programming language which gives a clear structure to programs and allows code to be reused, lowering development costs.
- C++ is portable and can be used to develop applications that can be adapted to multiple platforms.
- C++ is fun and easy to learn!
- As C++ is close to C# and Java.

Getting Started with C++

- To start your program development with C++, first you need to install two kinds of software on your

computer

1. Text Editor

- This will be used to type the code of your program.
- Example: Notepad, Code Blocks.

2. C++ Compiler

- Compilers are used to compile your source code and translate it into a machine code, a code that a computer can actually understand.

Sample code:

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Hello world!" << endl;
    return 0;
}
```

#include <iostream> – strands or standard input - output stream. It declares objects that control reading from and writing to the standard streams

using namespace std; – It means that we can use names for objects and variables from the standard library.

int main () – Defines the entry or the starting point of the C/C++ program code.

cout – used to display output to the screen. The "c" in cout stands for character, and "out" stands for output.

endl – used to insert a new line character and flush the stream.

return 0 – returns 0 means that the program will execute successfully and did what it was intended to do.

C++ Code Parts

The various parts of the given program consists of the ff:

- The header **<<iostream>>** contains everything we needed for handling input and output.
- The line **using namespace std;** tells the compiler to use the standard namespace. Namespaces are used to organize code into logical groups.
- The line **int main()** is the main function where program execution begins.
- The next line **cout << "This is my first C++ program.";** causes the message "This is my first C++ program" to be displayed on the screen.
- The next line **return 0;** terminates main() function and causes it to return the value 0 to the calling process.



Main

Program execution begins with the main function. `int main ()`

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Hello world!" << endl;
    return 0;
}
```

- `cout` access () indicates the beginning and end of the function, which can also be called the function's body. The information inside the brackets indicates what is the function does when executed.

Statements

A block is a set of logically connected statements, surrounded by opening and closing curly braces.

For example:

```
cout << "Hello world!" << endl;
return 0;
```

- You can have multiple statements on a single line, as long as you remember to end each statement with a semicolon, failing to do so will result in an error.

Getting the Tools

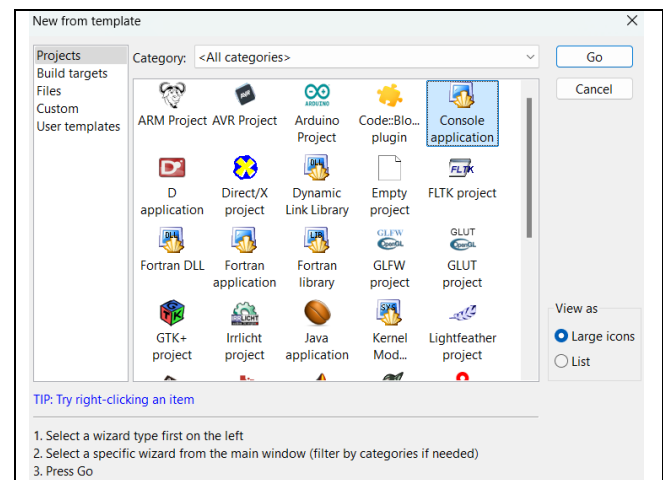
You need both of the following components to build C++ programs. can be used as on IDE.

1. Integrated Development Environment (IDE): Provides tools for writing source code. Any text editor
2. Compiler: Compiles source code into the final executable program. There are a number of C++ compilers available. The most frequently used and free available compiler is the **GNU C/C++** compiler.

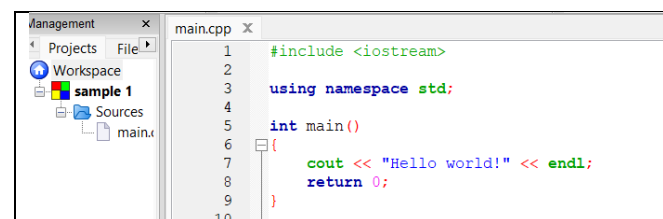
Various C++ IDEs and compilers are available. We'll use a free tool called Code: Blocks, which includes both an IDE and a compiler, and is available for Windows, Linux and MacOS. To download Code: Blocks, go to <http://www.codeblocks.org/>, Click the Downloads link, and choose "Download the binary release" Choose your OS and download the setup file, which includes the C++ compiler (For Windows, it's the one with mingw in the name).

How to create new project

- To create a project, open Code-Blocks and click "Create a new project" (or File > New > Project). This will open a dialog of project templates. Choose Console application and click Go.



- Make sure the Compiler is selected and click. Finish.
- GNU GCC is one of the popular compilers available for Code: Blocks.
- On the left sidebar, expand Sources. You'll see your project, along with its source files. Code-Blocks automatically created a main.cpp file that includes a basic Hello World program (C++ source files have .cpp, .c or .c extensions).



Printing a Text

Your first C++ Program

You can add multiple insertion operation after `cout`.

- `cout << "This" << "is" << "awesome";`

```
int main()
{
    cout << "This " << "is " << "Awesome!";
    return 0;
}
```

New Line

- The `cout` object does not insert a one break at the end of the output. One way to print two lines is to use the `endl` manipulator, which will put in a line break.



```
int main()
{
    cout << "Hello world!" << endl;
    cout << "I love when you cout me out";
    return 0;
}
```

- The endl manipulator moves down to a new line to print the second text.

- The new line character \n can be used as an alternative to endl, the backslash (\) is called an escape character and indicates a "special" character.

Examples:

```
int main()
{
    cout << "90 proof with J.cole \n";
    cout << "Smino and J.cole";
    return 0;
}
```

Results:

Comment

- Comments are explanatory statements that you can include in the C++ code to explain what the code is doing.
- The compiler ignores everything that appears in the comment, so none of that information shows in the result
- A comment beginning with two slashes (//) is called a single-line comment. The slashes tell the compiler to ignore everything that follows, until the end of the line.

Examples:

```
int main()
{
    // prints "90 proof with J.cole"
    cout << "90 proof with J.cole \n";
    // prints "Smino and J.cole"
    cout << "Smino and J.cole";
    return 0;
}
```

- When the above code is compiled, it will ignore the // prints "Hello world" statement and will produce the following

Multi-Line Comments

- Comments that require multiple lines begin with /* and end with */ You can place them on the same line or insert one or more lines between them.

```
int main()
{
    /* this is not necessarily needed*/
    cout << "90 proof with J.cole \n";
    /* one of one again!!*/
    cout << "Smino and J.cole";
    return 0;
}
```

If you write a wrong code segment, don't delete it immediately, put it into a multi-line comment, and then delete it. When you find the right solution.

Using Comments

- Comments can be written anywhere and can be repeated any number of times throughout the code. Within a comment marked with /* and */, // characters have no special meaning, and vice versa. This allows you to "nest" one comment type within the other.

Example:

```
int main()
{
    /* this is not necessarily needed*/
    cout << "90 proof with J.cole \n";
    cout << "Smino and J.cole"; //33 De Lorean//
    return 0;
}
```



Variables and Values

Variables

- are nothing, but reserved memory locations to store values.
- $x = 10$
- x – Variable
- 10 – Value
- Creating a variable reserves a memory location, or a space in memory for storing values. The compiler requires that you provide a data type for each variable you declare.
- C++ offers a rich assortment of built-in as well as user defined data types.
- Integer, a built-in type, represents a whole number value. Define integer using the keyword `int`.
- C++ requires that specify the type and the identifier for each variable defined.
- An Identifier is a name for a variable, function, module, or any other user-defined item. An identifier starts with a letter (A-Z or a-z) or an underscore (`_`), followed by additional letters, underscores, and digits (0 to 9).

For example, define a variable called `myVariable` that can hold integer values as follows:

```
int myVariable = 10;
```

- Defines all variables with a name and a data type before using them in a program. In cases in which you have multiple variables of the same type, It's possible to define them in one declaration, separating them with commas.

```
int a, b;  
// define two variables of type int
```

- A variable can be assigned a value and can be used to perform operations.
- For example, we can create an additional variable called `sum` and add two variables together.

```
int a = 30;  
int b = 15;  
int sum = a + b;  
// Now sum equals 45
```

Use the `+` operator to add two numbers

Let's create a program to calculate and print the sum of two integers.

```
#include <iostream>  
  
using namespace std;  
  
int main()  
{  
    int a = 30;  
    int b = 12;  
    int sum = a + b;  
  
    cout << sum;  
  
    return 0;  
}  
//Outputs 42
```

Always keep in mind that all variables must be delivered with a name and a description type before they can be used

Working with Variables

User Input

- To enable the user to input a value, use `cin` in combination with the extraction operator (`>>`). The variable containing the extracted data follows the operator.
- The following example shows how to accept user input and store it in the `num` variable:

```
int num;  
cin >> num;
```

Accepting User Input

- The following program prompts the user to input a number and stores it in the variable `a`:

```
int a;  
cout << "Please enter a number n\";  
cin >> a;  
cout << "Results" << a;
```

- When the program runs, it displays the message "Please enter a number", and then wait for enter a number and press Enter, or Return.
- The entered number is stored in the variable `a`.

The program will wait for as long as the user need to type



- You can accept user input multiple times throughout the program:

```
#include <iostream>

using namespace std;

int main()
{
    int a, b;
    cout << "Enter a number \n";
    cin >> a;
    cout << "Enter another number \n";
    cin >> b;
    cout << "Results of a" << a << "and b is," << b;
```

- Let's create a program that accepts the input of two numbers and prints their sum.

```
#include <iostream>

using namespace std;

int main()
{
    int a, b;
    int sum;

    cout << "Enter a number \n";
    cin >> a;
    cout << "Enter another number \n";
    cin >> b;
    sum = a + b;
    cout << "Sum is" << sum << endl;
```